

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Домашняя работа 5

Выполнил:

Черников Кирилл

Группа
J3210

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

1 Цель работы

Изучить основные команды менеджера зависимостей NPM и освоить базовые концепции фреймворка Vue.js 3: создание проекта через Vite, компонентный подход, реактивность, передача данных через пропсы и пользовательские события.

2 Требования задания

- Изучить основные команды NPM (`npm init`, `npm install`, `npm run`)
- Создать Vue-проект с помощью Vite
- Реализовать несколько компонентов, демонстрирующих возможности Vue.js
- Предоставить скриншоты компонентов и код приложения

3 Ход выполнения работы

3.1 Основные команды NPM

NPM (Node Package Manager) — стандартный менеджер пакетов для Node.js. Основные команды, использованные в работе:

Команда	Описание
<code>npm init</code>	Инициализация нового проекта, создание <code>package.json</code>
<code>npm install</code>	Установка всех зависимостей из <code>package.json</code>
<code>npm install <пакет></code>	Установка конкретного пакета в <code>dependencies</code>
<code>npm install -D <пакет></code>	Установка пакета в <code>devDependencies</code>
<code>npm run dev</code>	Запуск dev-сервера (скрипт из <code>package.json</code>)
<code>npm run build</code>	Сборка проекта для продакшена

3.2 Создание проекта

Проект создан вручную с использованием Vite и плагина `@vitejs/plugin-vue`. Файл `package.json`:

```
{
  "name": "dataforge-hw5",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": { "vue": "^3.4.27" },
  "devDependencies": {
    "@vitejs/plugin-vue": "^5.0.4",
    "vite": "^5.2.11"
  }
}
```

Конфигурация Vite (vite.config.js):

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'

export default defineConfig({
  plugins: [vue()],
})
```

Структура проекта:

```
vue-app/
  index.html
  vite.config.js
  package.json
  src/
    main.js
    App.vue
    assets/styles.css
    components/
      UserGreeting.vue
      CounterWidget.vue
      TaskList.vue
```

Точка входа src/main.js:

```
import { createApp } from 'vue'
import App from './App.vue'
import './assets/styles.css'

createApp(App).mount('#app')
```

3.3 Компонент UserGreeting — пропсы и события

Компонент принимает имя и роль через пропсы и отправляет событие `update:role` при смене роли через выпадающий список. Это демонстрирует однонаправленный поток данных и паттерн `v-model` на пользовательских компонентах.

```
<template>
  <div class="card">
    <h3>        , <strong>{{ name }}</strong>!</h3>
    <div class="role-tag">{{ role }}</div>
    <select :value="role"
      @change="$emit('update:role', $event.target.value)">
      <option>                </option>
      <option>                </option>
      <option>                </option>
    </select>
  </div>
</template>

<script setup>
defineProps({ name: String, role: String })
defineEmits(['update:role'])
</script>
```

3.4 Компонент CounterWidget — реактивность

Компонент демонстрирует реактивность Vue через `ref`. Значение счётчика меняется при нажатии на кнопки, интерфейс обновляется автоматически.

```
<template>
  <div class="card">
    <div class="counter-value">{{ count }}</div>
    <div class="btn-row">
      <button @click="count++">+</button>
      <button :disabled="count === 0" @click="count--">-</button>

      <button :disabled="count === 0" @click="count = 0">0</button>
    </div>
  </div>
</template>

<script setup>
import { ref } from 'vue'
const count = ref(0)
</script>
```

3.5 Компонент TaskList — директивы v-for и v-if

Компонент реализует список задач и демонстрирует:

- `v-for` — рендеринг списка задач
- `v-if` / `v-else` — условный рендеринг пустого состояния
- `v-model` — двустороннее связывание поля ввода и чекбоксов
- `computed` — автоматическое вычисление числа выполненных задач

```
<template>
  <div class="card">
    <form @submit.prevent="addTask">
      <input v-model="newTask" placeholder="..." />
      <button type="submit">Add Task</button>
    </form>

    <p v-if="tasks.length === 0">No tasks</p>

    <ul v-else>
      <li v-for="task in tasks" :key="task.id"
        :class="{ done: task.done }">
        <input type="checkbox" v-model="task.done" />
        <span>{{ task.text }}</span>
        <button @click="removeTask(task.id)">X</button>
      </li>
    </ul>
  </div>
</template>
```

```

        </li>
    </ul>
    <p>                : {{ doneCount }} / {{ tasks.length }}</p>
</div>
</template>

<script setup>
import { ref, computed } from 'vue'

const newTask = ref('')
const tasks = ref([
  { id: 1, text: '                Vue.js', done: true
    },
  { id: 2, text: '                ',
    done: false },
])
let nextId = 3

const doneCount = computed(
  () => tasks.value.filter(t => t.done).length
)

function addTask() {
  const text = newTask.value.trim()
  if (!text) return
  tasks.value.push({ id: nextId++, text, done: false })
  newTask.value = ''
}

function removeTask(id) {
  tasks.value = tasks.value.filter(t => t.id !== id)
}
</script>

```

3.6 Корневой компонент App.vue

Корневой компонент импортирует три дочерних компонента и передаёт данные через пропсы. Смена роли в `UserGreeting` обновляет состояние в родителе через `v-model`.

```

<template>
  <div class="app">
    <header class="app-header">
      <h1>DataForge                Vue.js</h1>
    </header>
    <div class="grid">
      <UserGreeting :name="userName" v-model:role="userRole" />
      <CounterWidget />
    </div>
    <TaskList />
  </div>
</template>

```

```

<script setup>
import { ref } from 'vue'
import UserGreeting from './components/UserGreeting.vue'
import CounterWidget from './components/CounterWidget.vue'
import TaskList from './components/TaskList.vue'

const userName = ref('')
const userRole = ref('')
</script>

```

4 Скриншоты

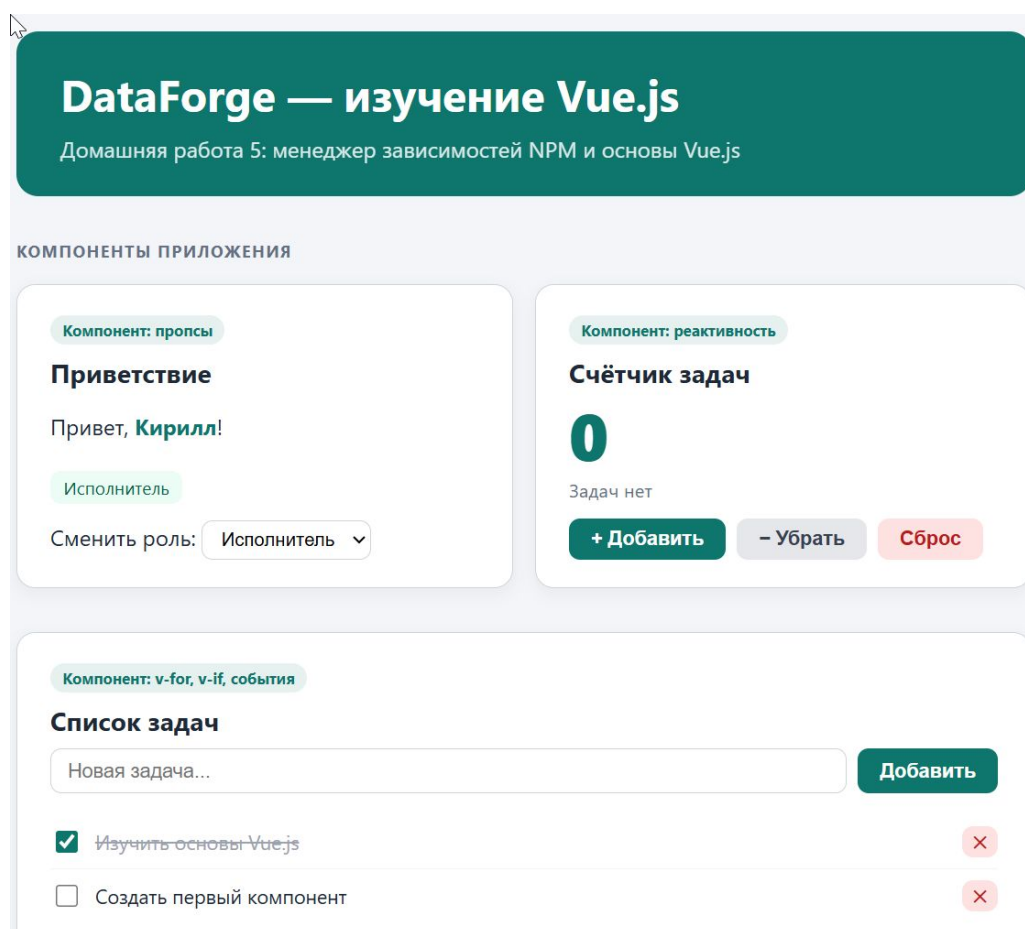


Рис. 1: Общий вид приложения: компоненты UserGreeting (пропсы, смена роли), CounterWidget (реактивность) и TaskList (v-for, v-if) на одной странице

5 Вывод

В ходе выполнения домашней работы изучены основные команды NPM и базовые концепции Vue.js 3. Создан Vue-проект на базе Vite с тремя компонентами, демонстрирующими: передачу данных через пропсы (UserGreeting), реактивность через

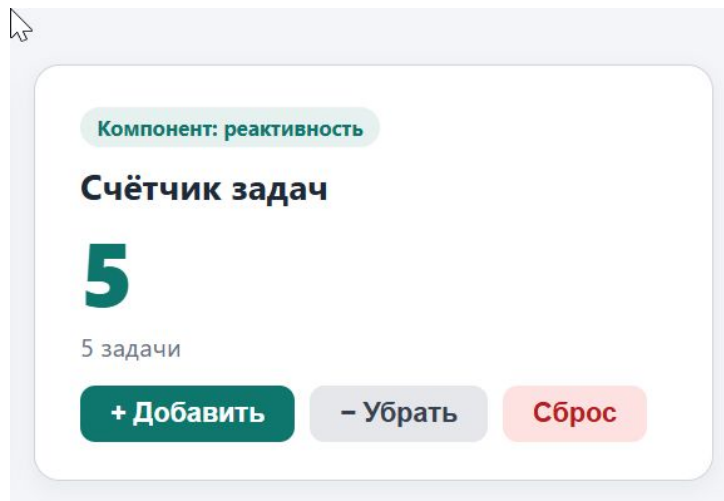


Рис. 2: Компонент CounterWidget — счётчик показывает значение 5 после пяти нажатий кнопки «+ Добавить»; кнопки «Убрать» и «Сброс» становятся активными

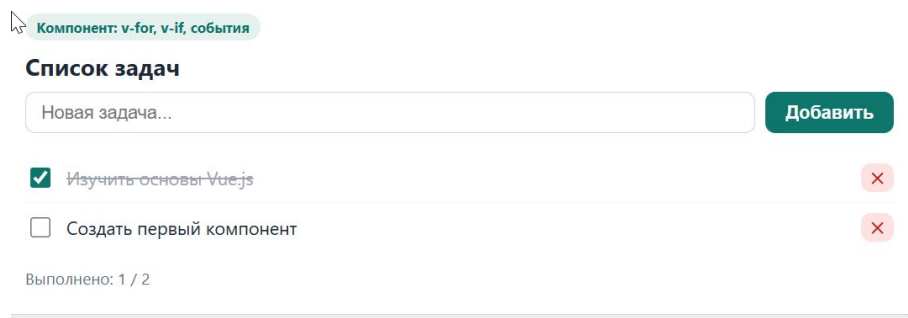


Рис. 3: Компонент TaskList — задача «Изучить основы Vue.js» отмечена как выполненная (зачёркнута через v-if/v-bind:class), счётчик «Выполнено: 1 / 2» обновляется реактивно через computed

ref (CounterWidget), условный рендеринг и рендеринг списков через v-if и v-for (TaskList). Все концепции, изученные в ДЗ5, применены в полноценном SPA в рамках ЛР3.